ECE420 Quartus Tutorial01: 2-bit Adder Design GLaw

Schematic Design Entry Method and Simulation with Modelsim

Notation:

To simplify what one should click to perform a task, the following form of notation will be used:

Menu item → Menu item → … → Menu item

Example:

File → Project Wizard means "on the File menu, click New Project Wizard"

Objective

To learn to use Quartus to develop and simulate a structured 2-bit adder using Schematic Design Entry Method

Development Tools

Quartus 11.0

Hardware

DE2 board, USB Blaster, and USB cable.

References

Altera Quartus Handbooks Volume 1, 2, and 3 available at www.altera.com or at the course website www.csun.edu/~glaw

Launch Quartus II Software

1. Launch the Quartus II software.
2. File → Project Wizard (on the File menu, click New Project Wizard)
3. New Project Wizard: Introduction window appears. It shows you the five steps you may need to go through in setting up a new project. You have nothing to do in this window, hence click **Next**.
4. New Project Wizard: Directory, Name, Top-Level Entity window appears.

   a. Enter the working project directory in your Z drive, for example, z:\...\Tutorial2

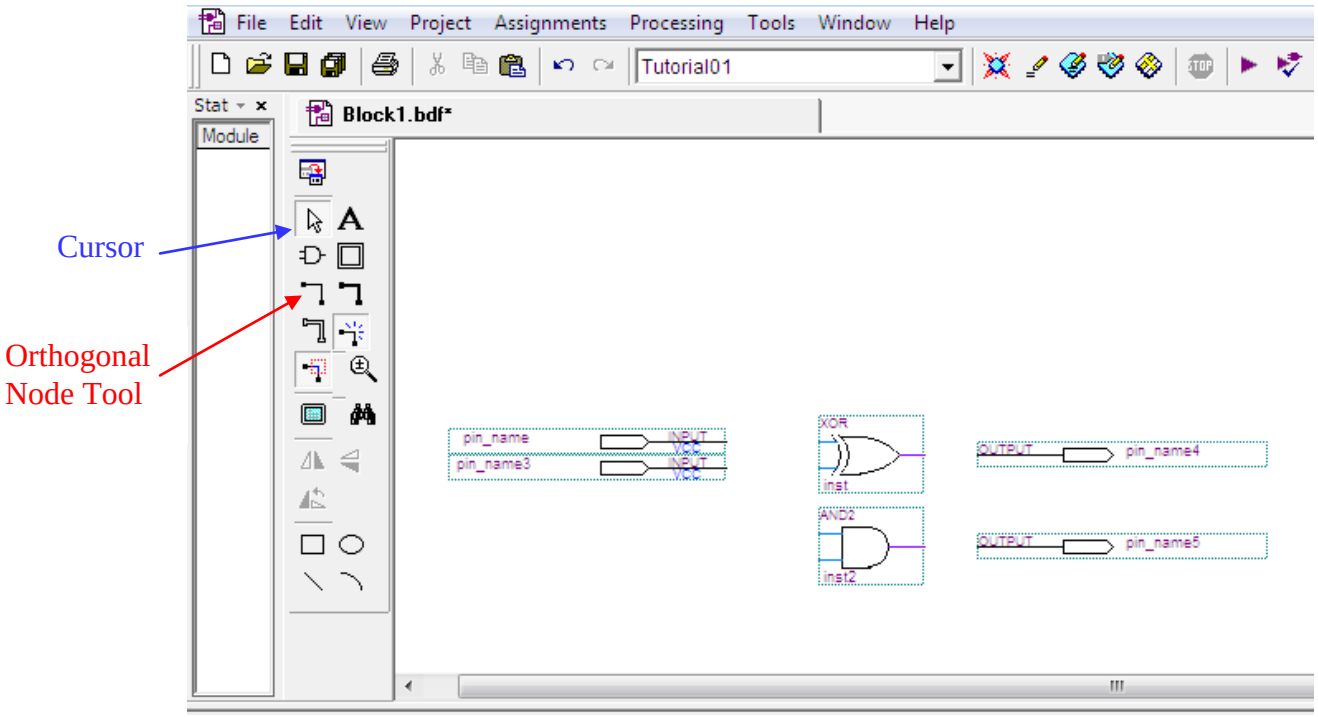
   b. Enter project name: Adder2bit. As you type Adder2bit name, Top-Level Entity name will be automatically filled with Adder2bit.

   c. Click next. When asked to create z:\...\Tutorial2 directory, click **Ye**s.
5. New Project Wizard: Add Files window appears. You have no files to add now, but need to add files later. Hence, just click **Next**.

6. New Project Wizard: Family & Device Settings window appears.
   Scroll up/down the family choices and choose Cyclone II family. Scroll up/down the available devices and choose EP2C35F672C6. Leave other options as they are. Click **Finish**.

Create a Half Adder

1. File → New. New file option window appears.
2. Select Block Diagram/ Schematic File. Click OK. BDF Editor window appears.
3. Double click anywhere on the window. Select Symbol window appears.
4. Type xor in the Name box. Click Ok. Drag the XOR symbol and put it the middle of the window.
5. Repeat steps 3 and 4 but with and2 symbol name to put a 2-input AND symbol at the bottom of the XOR symbol.
6. Input ports and output ports are symbols with names "input" and "output", respectively. Thus, repeat steps 3 and 4 with symbol name "input" and "output" to put two input ports and two output ports on the window. Click and drag the symbols on the window to match the BDF diagram shown below.



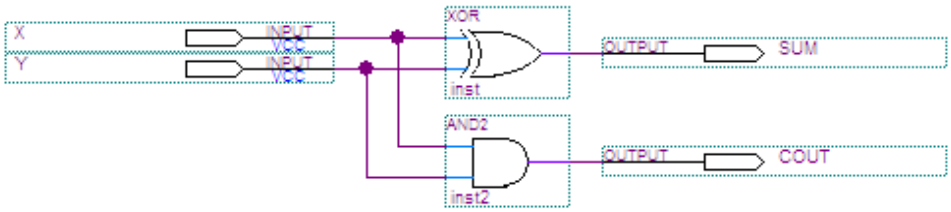
7. Click the orthogonal node tool in the list of tool on the left of the BDF window (see above). To connect point A to point B, position the cursor on point A, click and hold the left button of the mouse, drag the line and drop it at point B.

   Example: To connect input symbol to the XOR symbol input, position the cursor on the right end point of the input symbol, click and hold the left mouse button, drag the line and drop it on the input of the XOR symbol.

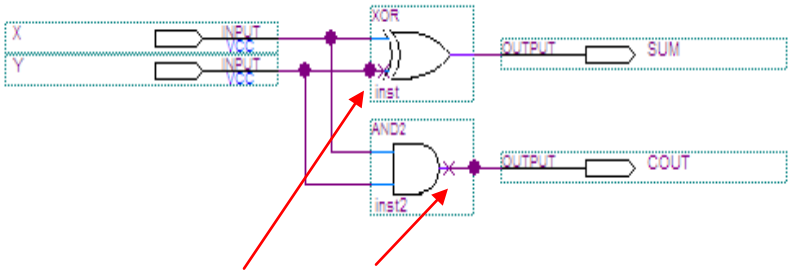
   Follow the steps above connect the input symbols and the output symbols to the XOR and AND symbols.

8. Click the cursor in the list of tool on the left of the BDF window (see above). Double click the input symbol and change the input name to X. Double click the other input symbol and change the input name to Y. Repeat the steps to name the output names to SUM and COUT, respectively.

   Now, your bdf file must look similar to the one shown below:



   To help you in identifying erroneous line connection, an example is shown below:



   The errors could be identified as “x” on the lines. If you see these “x”s, delete the lines and redraw them.

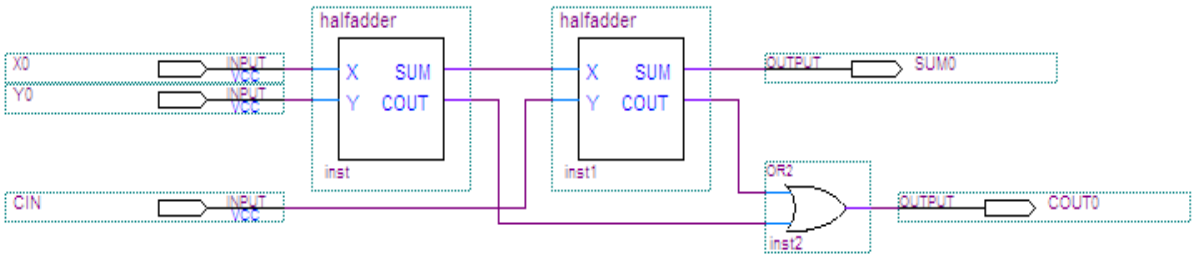
9. Save the bdf file as halfadder.bdf.

10. Add halfadder.bdf to project. Project → Add Current File to Project.

11. File → Create/Update → Create Symbol Files for Current File. Create symbol file window appears. Note that the filename shown is “halfadder”. Click Save. An acknowledgement that the halfadder symbol has been created appears. Click Ok.

Create a 2-bit adder using two halfadders and one OR gates.

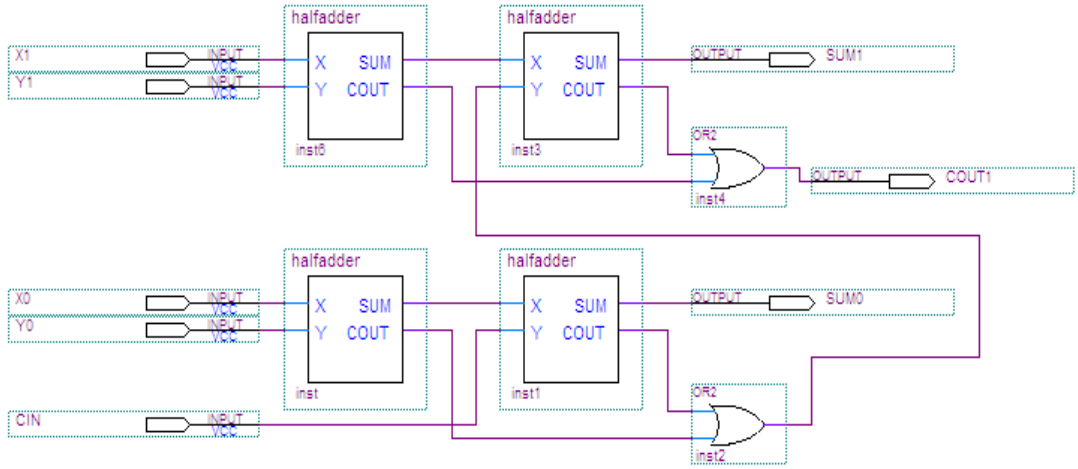
1. File → New. New file option window appears.
2. Select Block Diagram/ Schematic File. Click OK. BDF Editor window appears.
3. Double click anywhere on the window. Select Symbol window appears.
4. Expand the Project folder in Libraries window, click halfadder, and click Ok. Put the halfadder symbol in the middle of the bdf window.
5. Repeat steps 3 and 4 to put another halfadder symbol on the bdf window.
6. Put a 2-input OR symbol (OR2) on the bdf window. You should know how to do this by now. If not, review the previous steps on how to put an AND symbol on the bdf window.
7. Put three input symbols and two output symbols on the bdf window and connect them to the respective inputs and outputs. Rename the input symbols and output symbols as shown on the bdf diagram below:



Your fulladder bdf file must look similar to the one shown above. If there is a mismatch, go back to the previous steps to fix the errors.

8. You could have followed steps 3 to 7 to create another one-bit fulladder. But you would save time by using the copy and paste facilities. Select the entire fulladder you have created earlier by putting the cursor on the left hand upper corner, left click and drag the selection window to cover all symbols in the fulladder. Edit → Copy.
9. Move the cursor to a position above and far from the fulladder and paste the selection. Position this new fulladder (bit 1) so that it lines up with the fulladder (bit 0) below.
10. Delete Cout0 symbol for fulladder (bit 0) and delete CIN symbol for fulladder (bit1) but leave the lines intact. Connect the lines left open after deleting Cout0 and CIN.

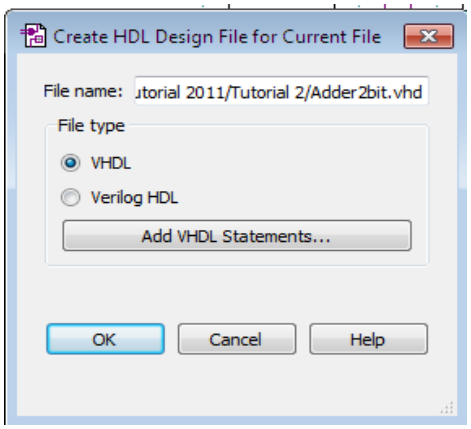
Your 2-bit adder bdf file must look similar to the one shown below. If there is a mismatch, go back to the previous steps to fix the errors.



11. Save your 2-bit adder file and name it Adder2bit.bdf. Note that the top design name must be the same as the project name. Here, the top design is a 2-bit adder and our project name is Adder2bit.

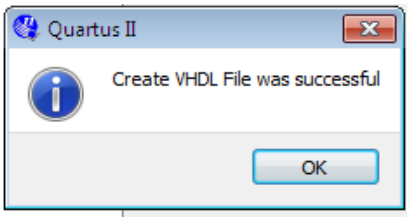
Convert BDF File to VHDL File

1. Ascertain that the Adder2bit.bdf is currently active (focused). File → Create/Update → Create HDL Design File from Current File. "Create HDL Design File from Current File" window appears.

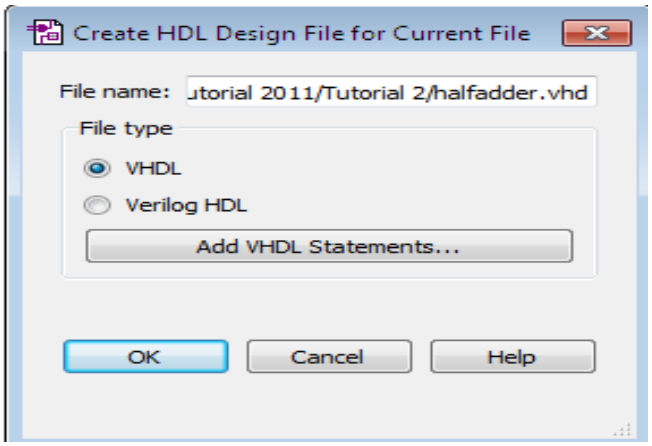


2. Choose VHDL file type and use the default file name Adder2bit.vhd. Click OK.

3. After the HalfAdder.vhd has been successfully created, click OK.

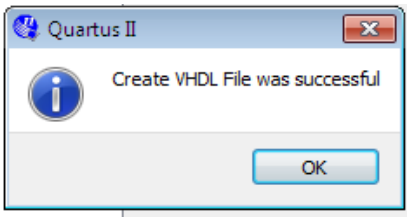


4. Ascertain that the halfadder.bdf is currently active (focused). File → Create/Update → Create HDL Design File from Current File. "Create HDL Design File from Current File" window appears.



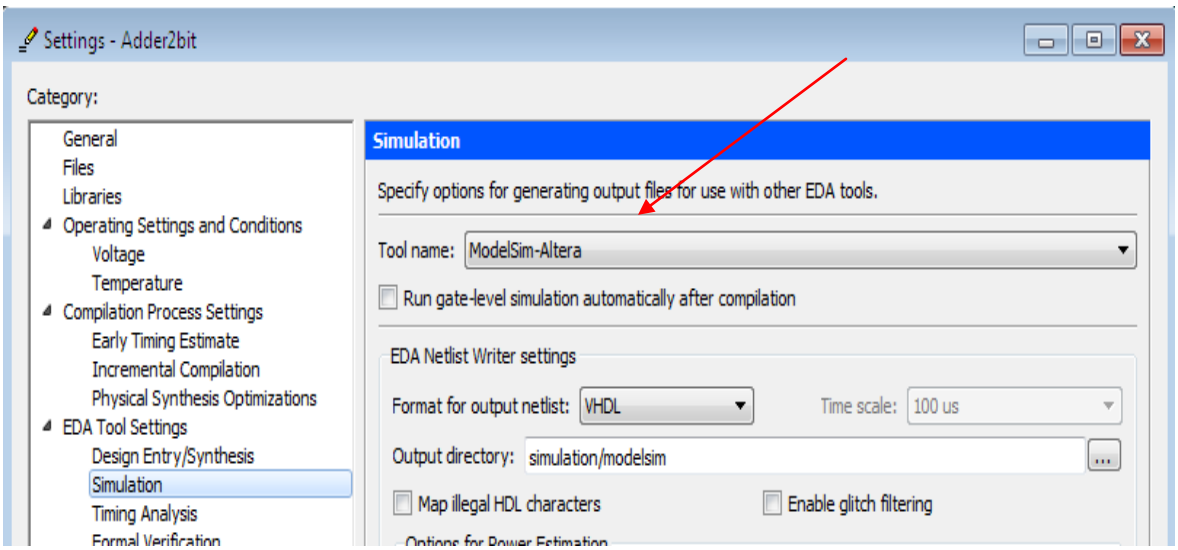
5. Choose VHDL file type and use the default file name halfadder.vhd. Click OK.

6. After the halfadder.vhd has been successfully created, click OK.

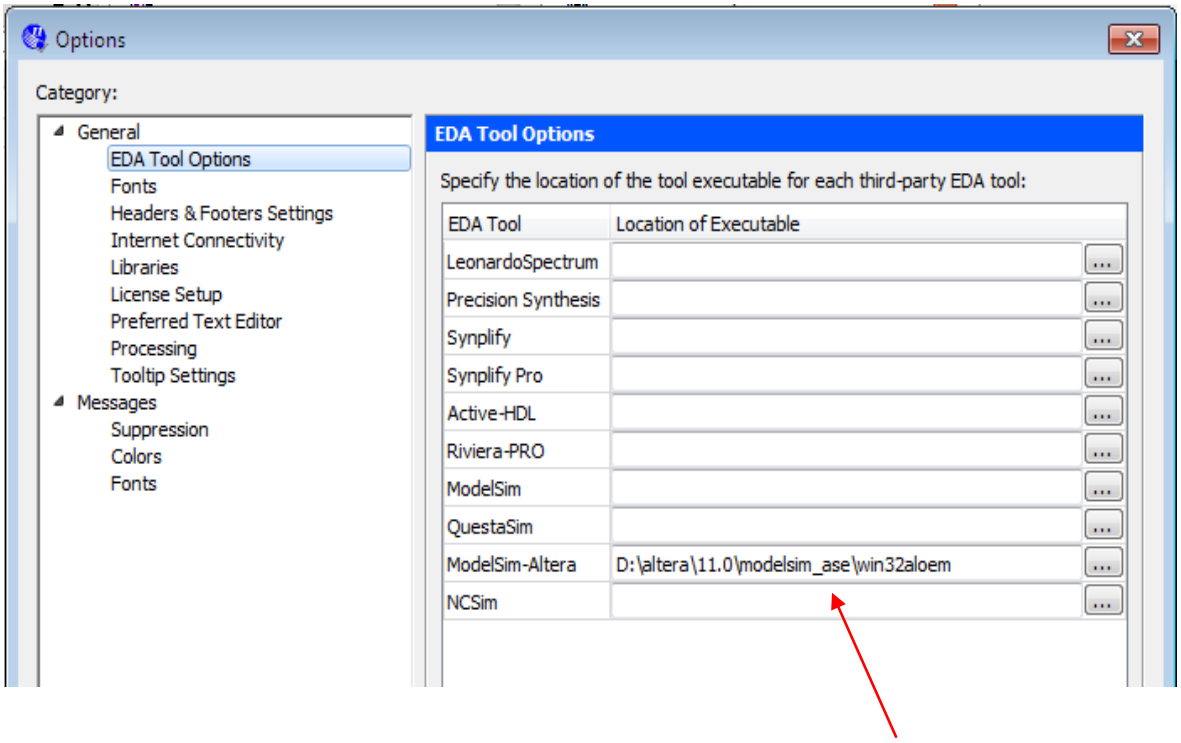


Set Quartus to Use ModelSim Simulator

1. Assignments → Settings. Adder2bit settings window appears.
2. Under EDA Tool setting, click simulation. Browse Tool Name and choose Modelsim-Altera. Leave the other fields at the default values and click OK.



3. Tools → Options. Tools Option window appears. Under Category /General, select EDA Tool Options. In the Modelsim-Altera field, browse the folders and locate win32aloem folder for the Modelsim executable file. Click OK.



Compile the 2-bit adder

Processing → Start Compilation. If there are any errors in the message window, fix the errors and compile the design again. For this tutorial, you may ignore the warnings.

You may leave Quartus open or minimize it. As in any design, the first simulation may not give the correct result, and you may need to use Quartus again to correct the design flaw.

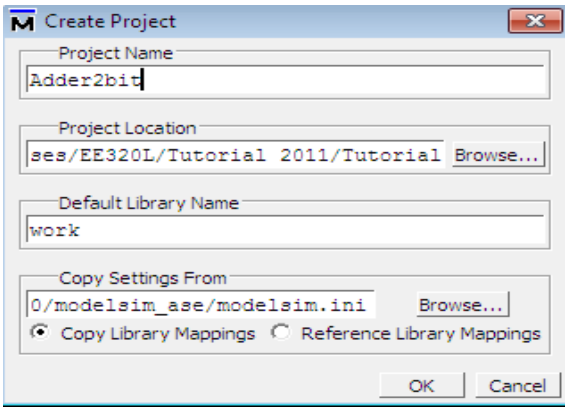
Simulate the 2-bit adder (Functional Simulation)

Note: Since you have not learned VHDL yet (ECE420), you are bound to simulate the design using waveform vectors instead of VHDL Testbench. As a note, simulating a design using VHDL Testbench is the standard procedure in the industry.

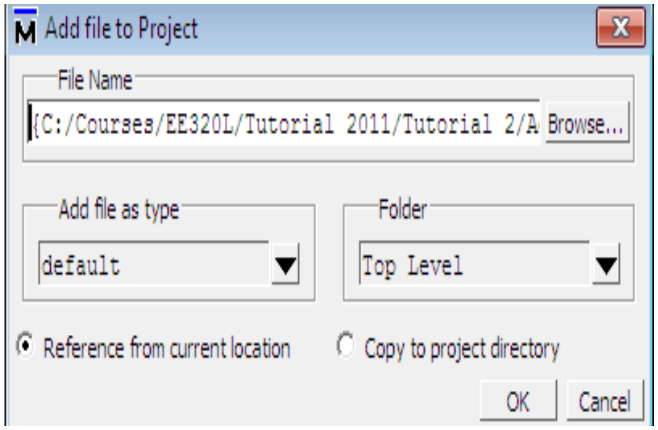
1. Launch Modelsim. When the welcome to version 6.6d window appears, click jumpstart.

2. When Welcome to Modelsim appears, click Create a Project. You can also create a new project later by File → New → Project.

3. Enter the Project name "Adder2bit" and project location "z:\...\Tutorial2" (the same project location specified in Quartus earlier). You may also use the browse feature to select the folder. Click OK.



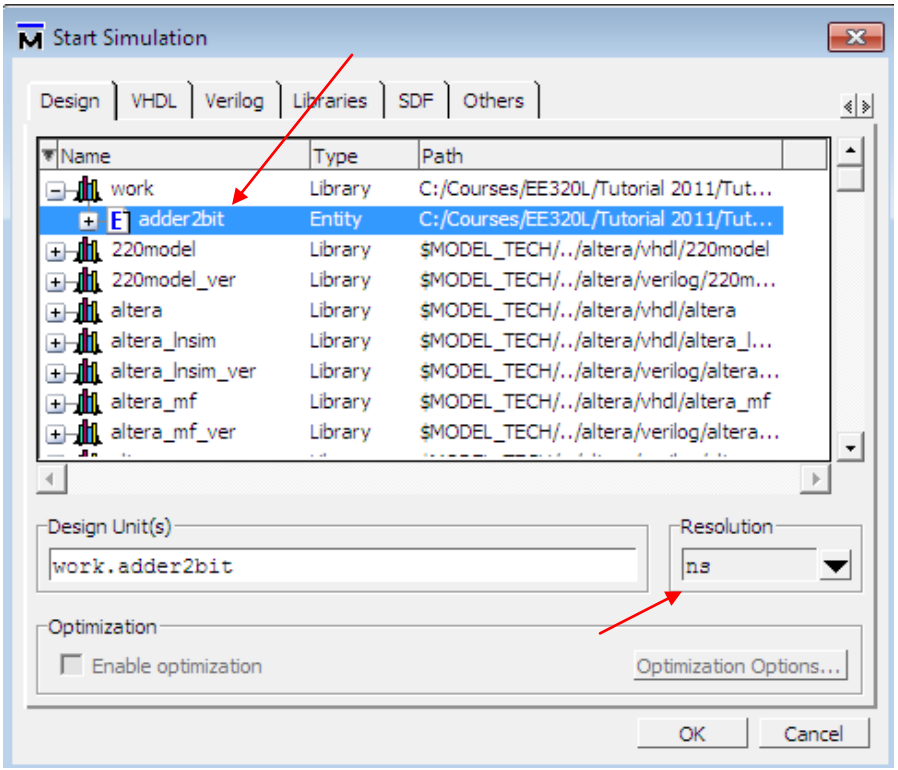
4. When Add Item to Project window appears, click Add Existing File. Browse to Adder2bit.vhd file created in Quartus. Click OK.



5. When Add Item to Project window appears again, click Add Existing File once more. This time browse to halfadder.vhd. Click OK and close Add item to Project window.

6. Compile → Compile All. After the compilation, the Compile of Adder2bit.vhd was successful message should appear in the transcript (message) pane.

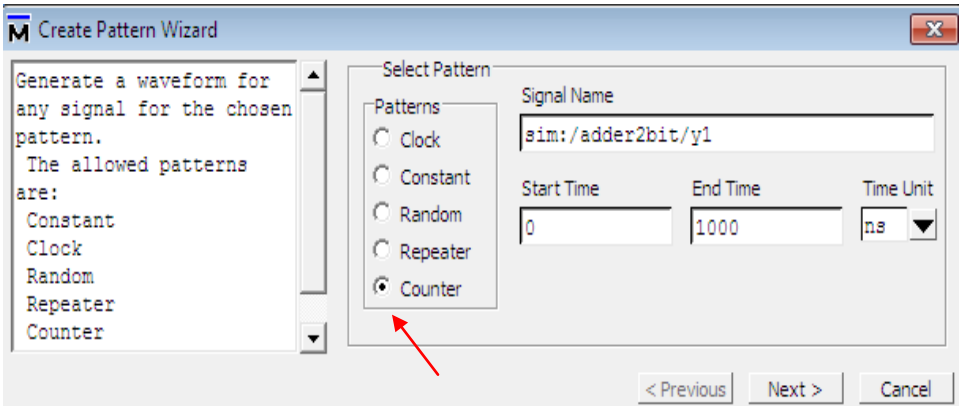
7. Simulation → Start Simulation. In Start Simulation window, expand work library and select Adder2bit. Choose ns resolution. Click OK.



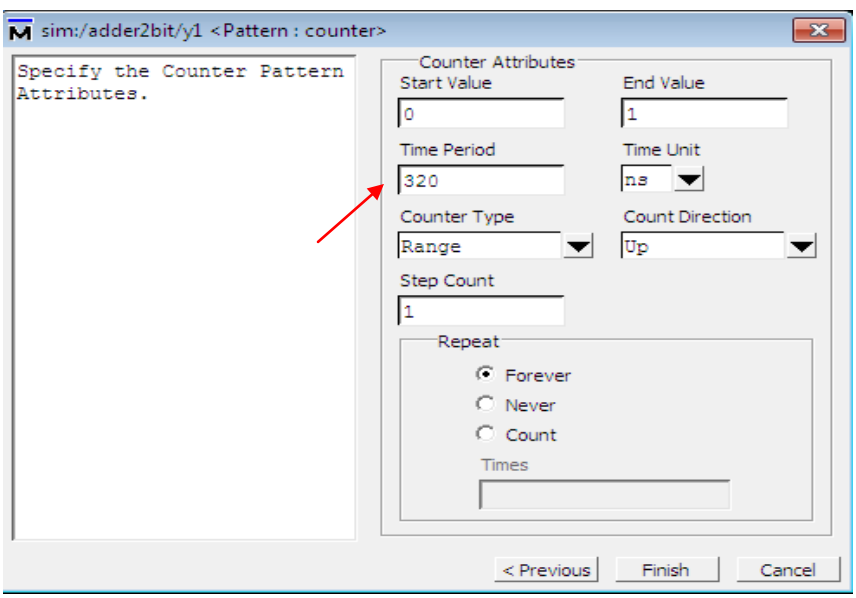
8. View →Object and View Wave. To view Objects and waveforms.

9. Note: Adder2bit has 5 inputs: y1, y0, x1, x0, and cin where y1 is the most significant bit and cin is the least significant bit. Hence, y1 has the weight of 2^4 and cin has the weight of 2^0.

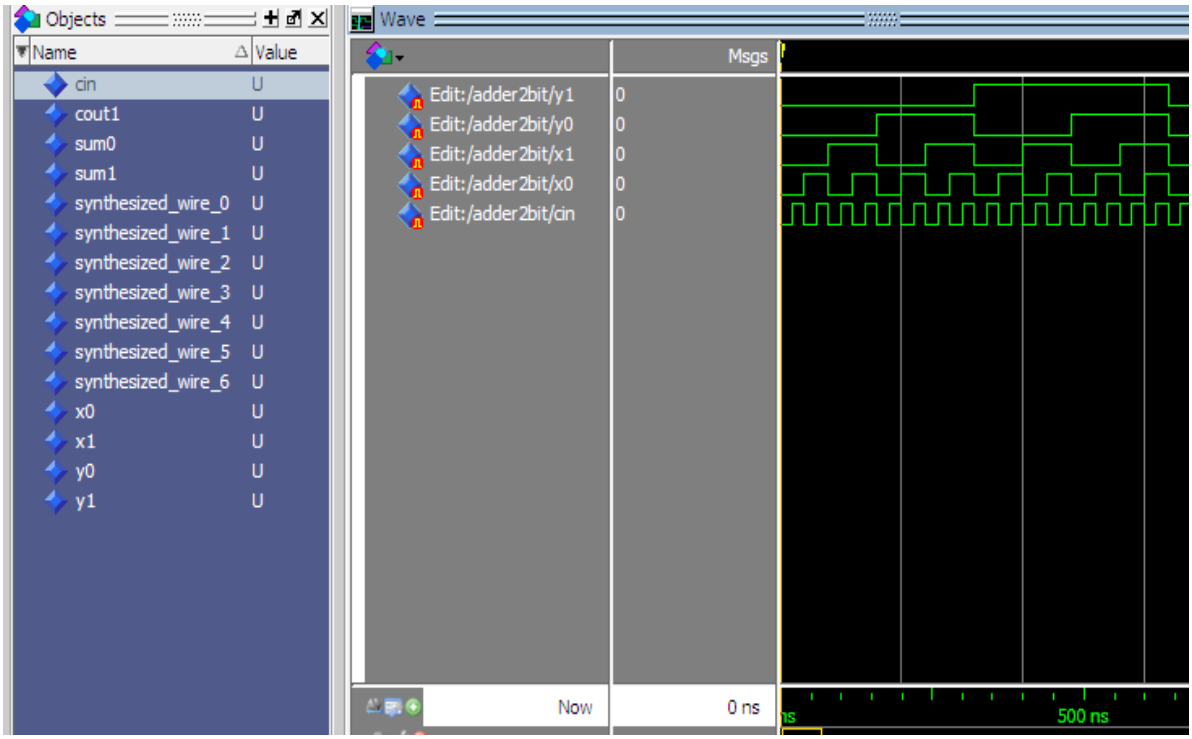
Right click on y1 signal in Object pane and click Create Wave. In Create Pattern Window, select Counter pattern. Click Next.



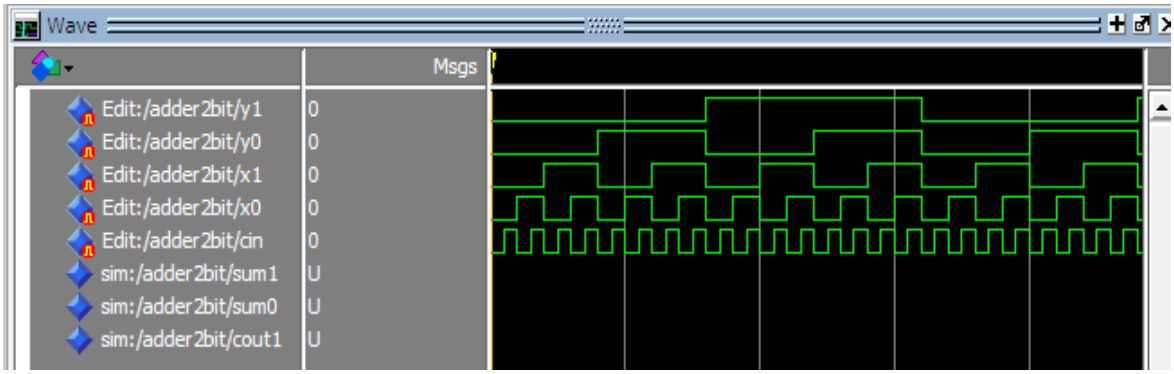
10. In the next window, set Period to 320 ns. Click Finish.



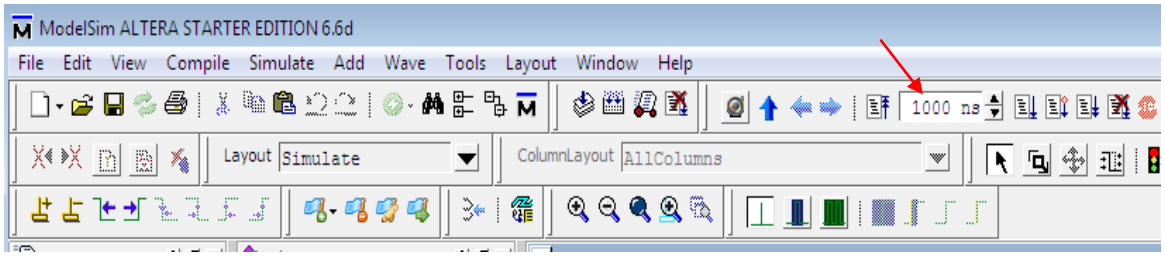
11. Repeat Steps 8 and 9 for the y0, x1, x0, and cin signals in the Object pane. But use 160 ns counter period for y0 because y0 is the 2^3 order signal; use 80 ns counter period for x1 because x1 is the 2^2 order signal; use 40 ns counter period for x0 because x0 is the 2^1 order signal; use 20 ns counter period for cin because cin is the 2^0 order signal. At this point, the wave pane should look like the one below:



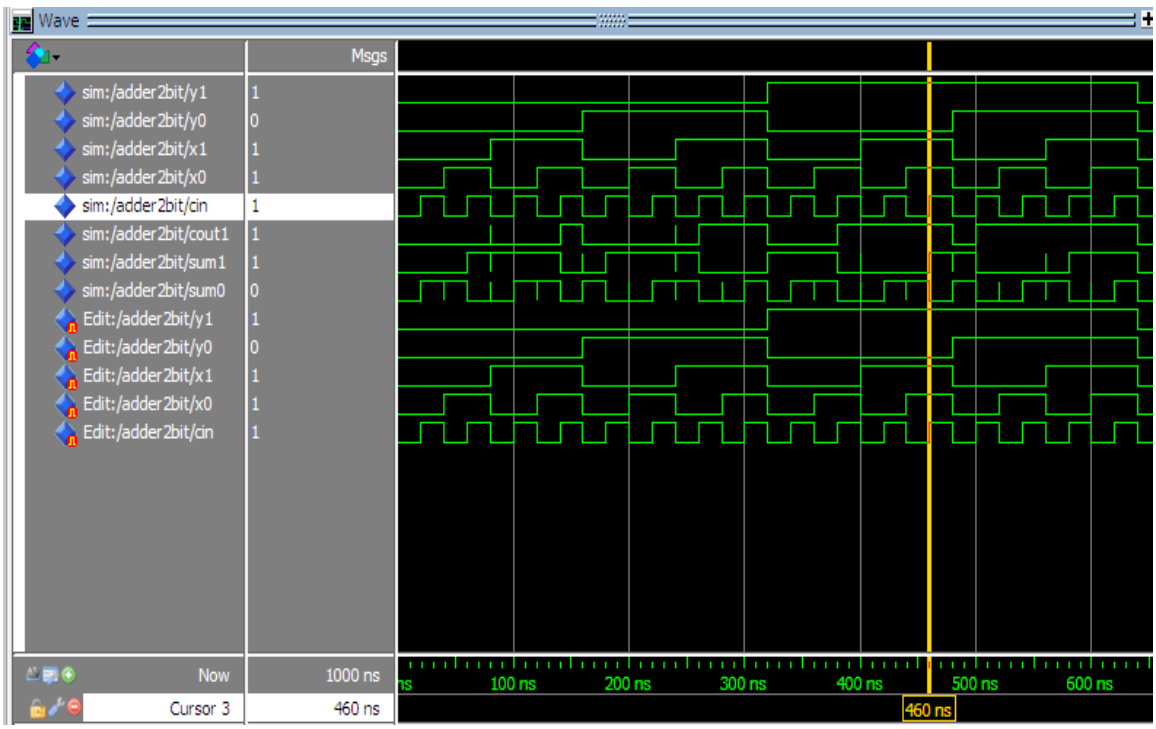
12. Right click on SUM1 signal in Object pane and Add → to Wave → Selected Signals. Repeat this step for the SUM0 and COUT1 signals. The wave pane should look like the one below:



13. You can set the simulation end time in top menu bar. As shown, it is now set to 1000 ns. For the Adder2bit, you only need 640 ns simulation time to cover all possible outcomes for the five combinations of y1, y0, x1, x0, and cin input combinations.



14. To run the simulation, Simulate → Run → Run-All. The simulation result is shown in the wave pane:

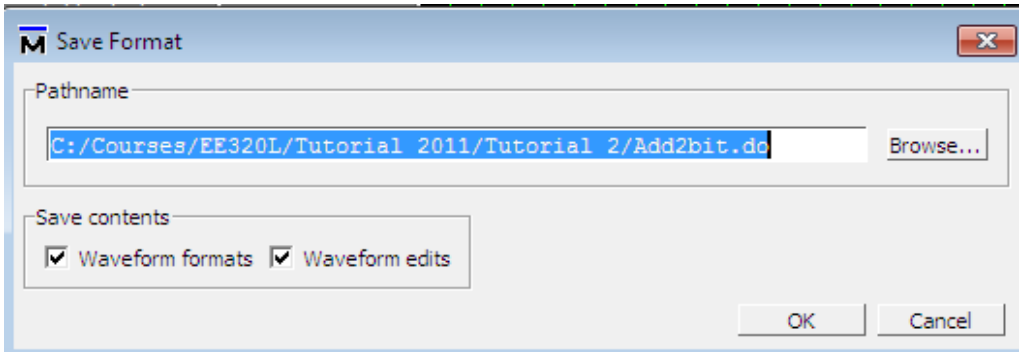


Zoom-in the wave window as many times as necessary so that you could convince yourself that there are no delay between input signals and outputs signals. Also, note that the simulation outputs are logically correct.

As shown in the simulation output, at time 460 ns, y1y0 + x1x0 + cin = 10 + 11 + 1 = 110. In other words, 2 + 3 + 1 = 6.

Note: If your simulation output does not show the signals in the same order as above, you can click the signal and drag it to any position as you wish.

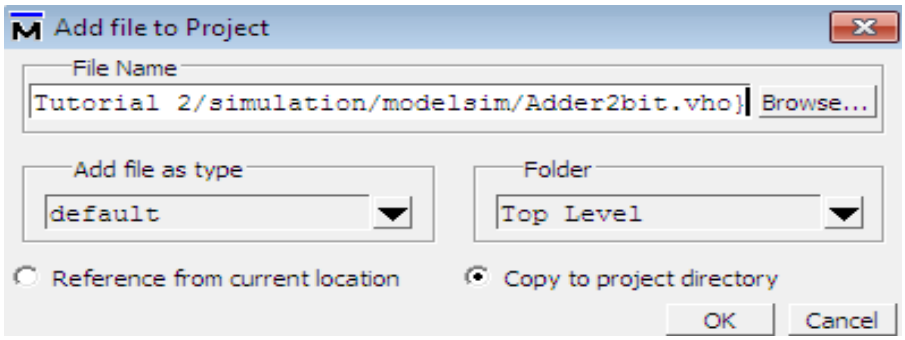
15. Save the simulation output and name it adder2bitFunctional.do



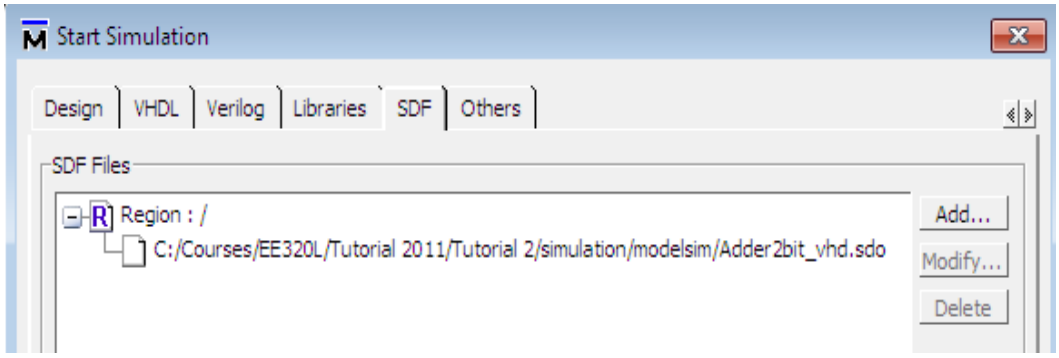
Simulate the half adder (Timing Simulation)

The difference in functional simulation and Timing simulation is that the Timing simulation needs adder2bit.vho to supply the delay information. Quartus generates adder2bit.vho and you need to include it in the project. The following steps will show you how to do it.

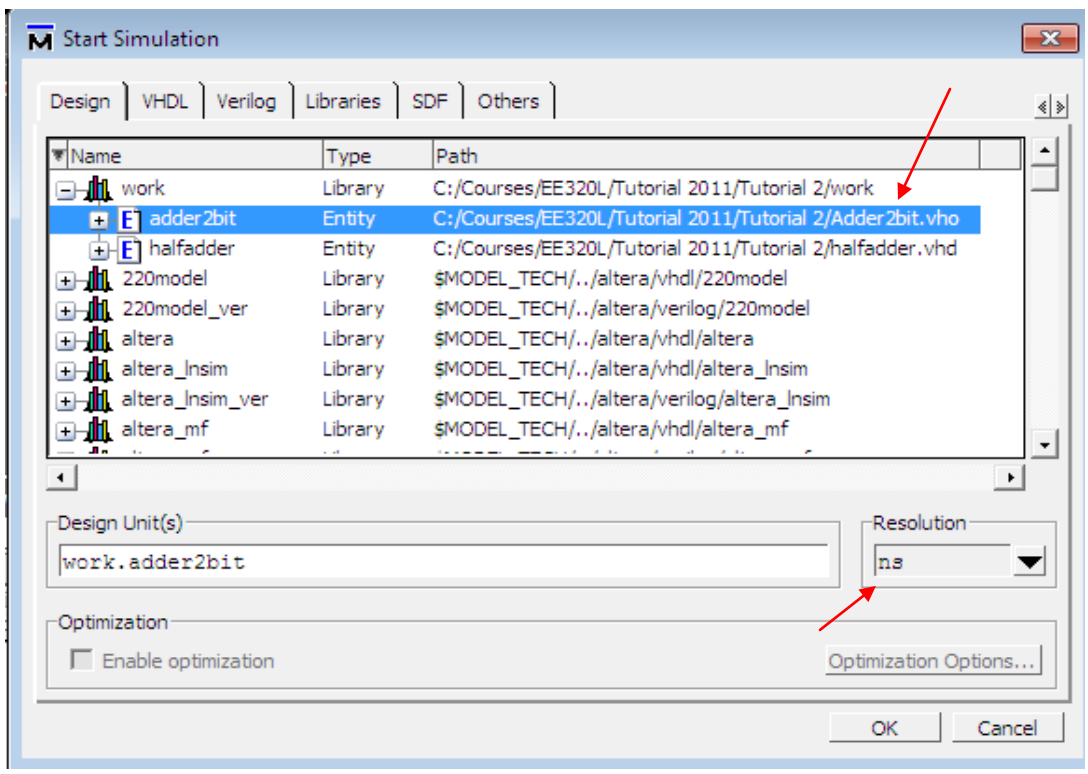
1. Open the project pane if it is not already open Right click in the project pane and Add to Project → Existing File. The Select a File to Add window appears. In this window, browse to simulation/Modelsim folder and select adder2bit.vho. Mark Copy to project directory and click OK.



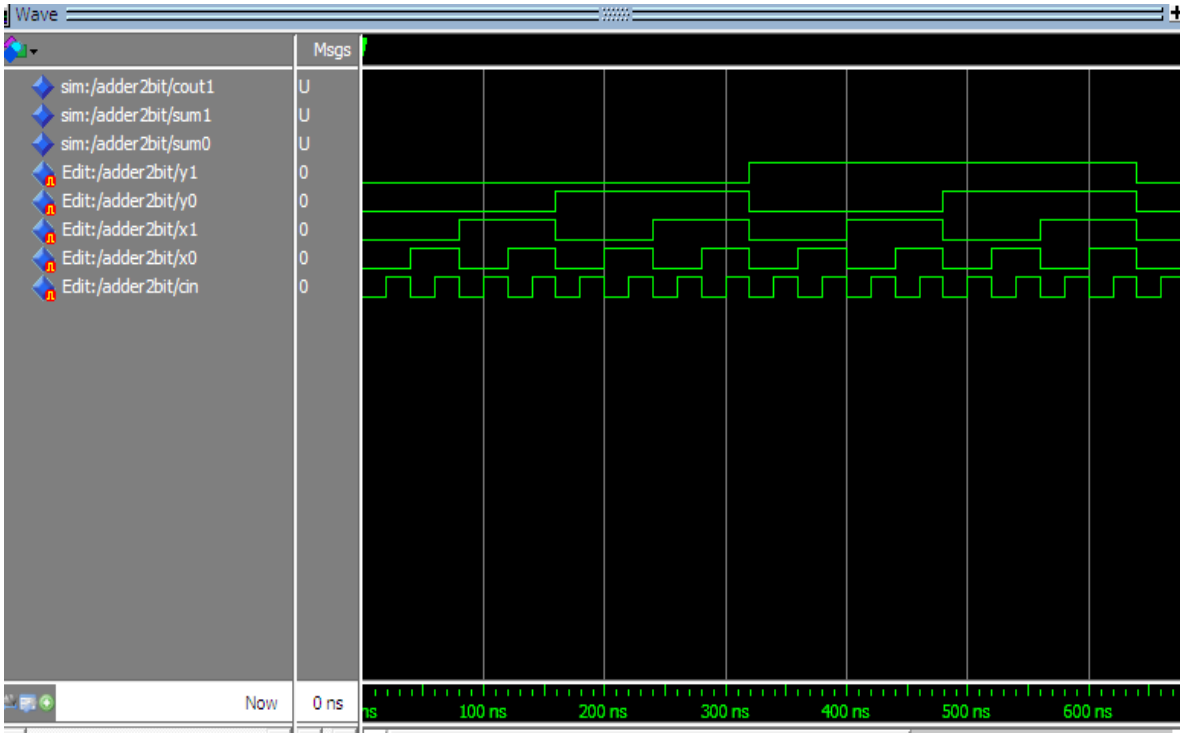
2. Right anywhere in the project pane. Compile → Compile All. If adder2bit.vho is not Top design, the compiler may give error messages. In this case, Compile → Compile Order. Click Auto Generate and click OK. Recompile all files: Compile → Compile All.
3. Simulate → Start Simulate. Start Simulation window appears. In this window, click SDF tab and then click add. Browse to simulation/Modelsim folder and select adder2bit_vhd.sdo file.



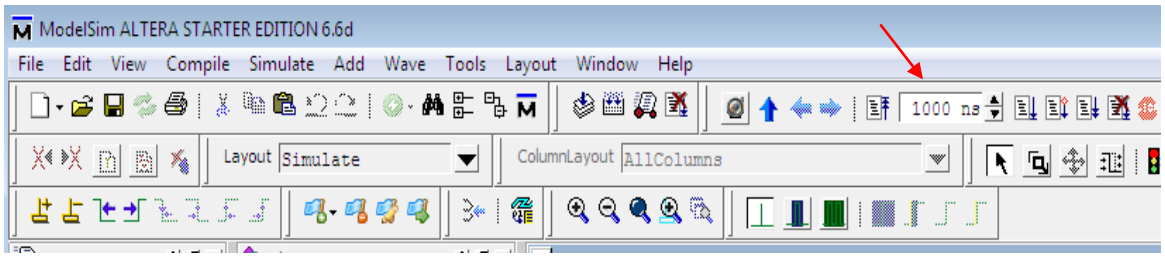
4. Click Design tab, expand work library and select adder2bit_vhd.sdo file and select ns resolution. Click OK. SDF. Backannotation Successfully Completed message should appear in the Transcript pane.



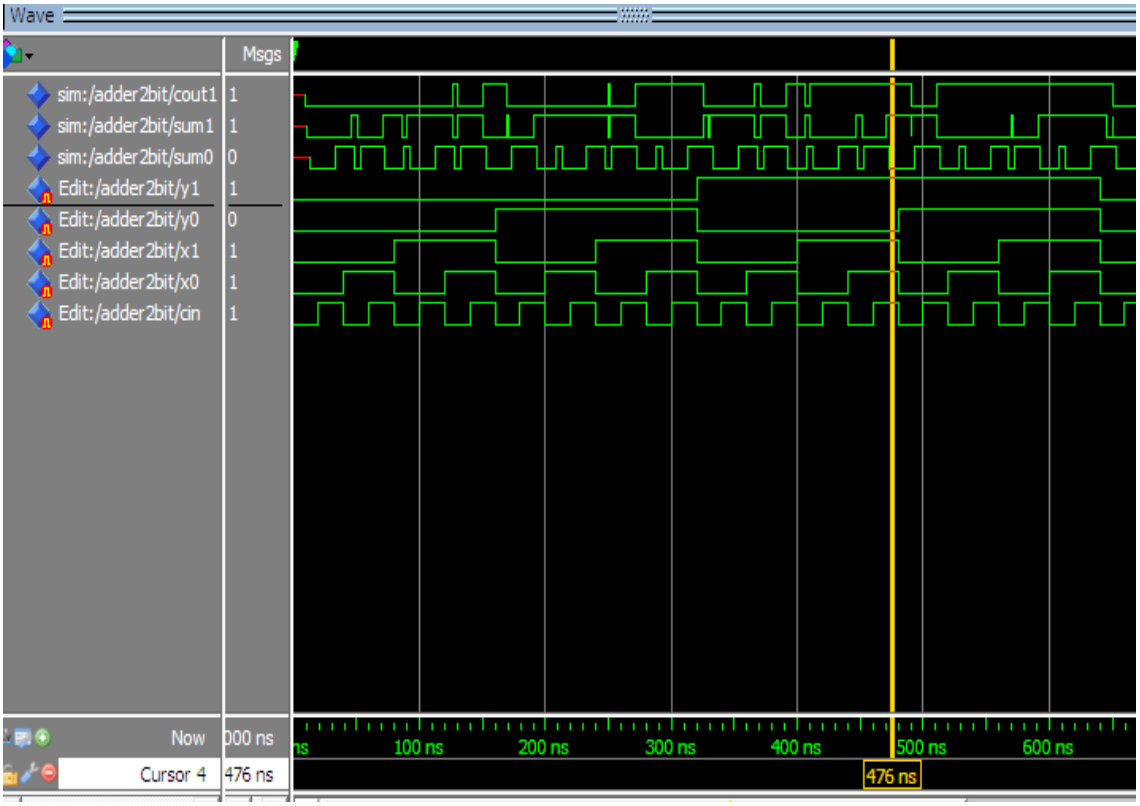
5. Follow Steps 7 to 11 in Functional Simulation section to set the input and output waveforms. If you do it correctly, wave pane should look like below. Another option is that you load the adder2bit.do you saved for functional simulation. File → Load. The Load option, however, may load extra signals on the wave window and you may need to delete and re-arrange the signals.



16. You can set the simulation end time in top menu bar. As shown, it is now set to 1000 ns. For the Adder2bit, you only need 640 ns simulation time to cover all possible outcomes for the five combinations of y1, y0, x1, x0, and cin input combinations.



17. To run the simulation, Simulate → Run → Run-All. The simulation result is shown in the wave pane:



18. Zoom-in the wave window as many times as necessary so that you could convince yourself that there are no delay between input signals and outputs signals. Also, note that the simulation outputs are logically correct.

19. As shown in the simulation output, at time 460 ns, y1y0 + x1x0 + cin = 10 + 11 + 1 = 110. In other words, 2 + 3 + 1 = 6. Note that the result is available at 476 ns which is delayed 16 ns from 460 ns.

    Note: If your simulation output does not show the signals in the same order as above, you can click the signal and drag it to any position as you wish.

20. Save the simulation output and name it Adder2bitTiming.do